INCOME AND EXPENSE TRACKER

Final Design Document, v1.0

Abstract

Design guide for a locally hosted web application to allow users to track and overview income and expenses of any number of their financial accounts.

Ezri King

3/12/2026

Amarillo College | Advanced Object-Oriented Programming ITSE-2357

Contents

Executive Summary 1
Introduction 1
Functional and Non-Functional Requirements 1
User Stories 1
Use Case Descriptions 2
UI Mock-Ups 2
Dashboard 2
Transactions 3
Accounts 3
Account Details 4
Use Case Diagram 5
Class Diagram 6
Sequence Diagram 7
Activity Diagram 7
State Diagram 8
Programs 8
Conclusion 11

Document Revision History

Version	Date	Author	Comments
1.0	03/12/2026	Ezri King	Created initial version.

Executive Summary

This document presents the design of a personal monetary tracking application that allows users to manage, filter, and view summaries of financial accounts. The app is modeled using object-oriented principles and UML diagrams to illustrate the structure and the component relationships. Core elements of the design include classes representing accounts and transactions with supporting management and display classes. The goal of the design is to create a clear structure, separating responsibilities and establishing architecture to implement during development.

Introduction

This document presents design documentation for a monetary tracking app. The document includes UML class diagrams and written descriptions that define the main application components and relationships between them. These diagrams illustrate the application structure and class interactions.

The application being designed allows users to track transactions, review account dashboards, and filter transaction history. This design includes classes that represent accounts and transactions, along with support components that manage and display app information. These elements define the structure of the proposed application.

Functional and Non-Functional Requirements

Functional	**Non-Functional**
Transaction Management The web application should enable users to add, edit, and delete entries for income and expenses. The web application must support multiple account histories within the same profile. The web application should summarize transaction data.	**Security** The web application should protect against common web vulnerabilities such as XSS and SQL injections. The system must have clearly separated frontend, backend, and database layers. (MVC) **Transaction Management** The web application should be able to manage a large history of entries without crashing or loading for more than 10 seconds. **Interface** The web application must be responsive.

User Stories

As a user, I want to have a summary of my accounts so I can see my financial wellbeing at a glance.

As a user, I want to add multiple accounts so I can include my spendings and savings in one app.

Use Case Descriptions

User will be able to add, edit, and delete income and expense entries.

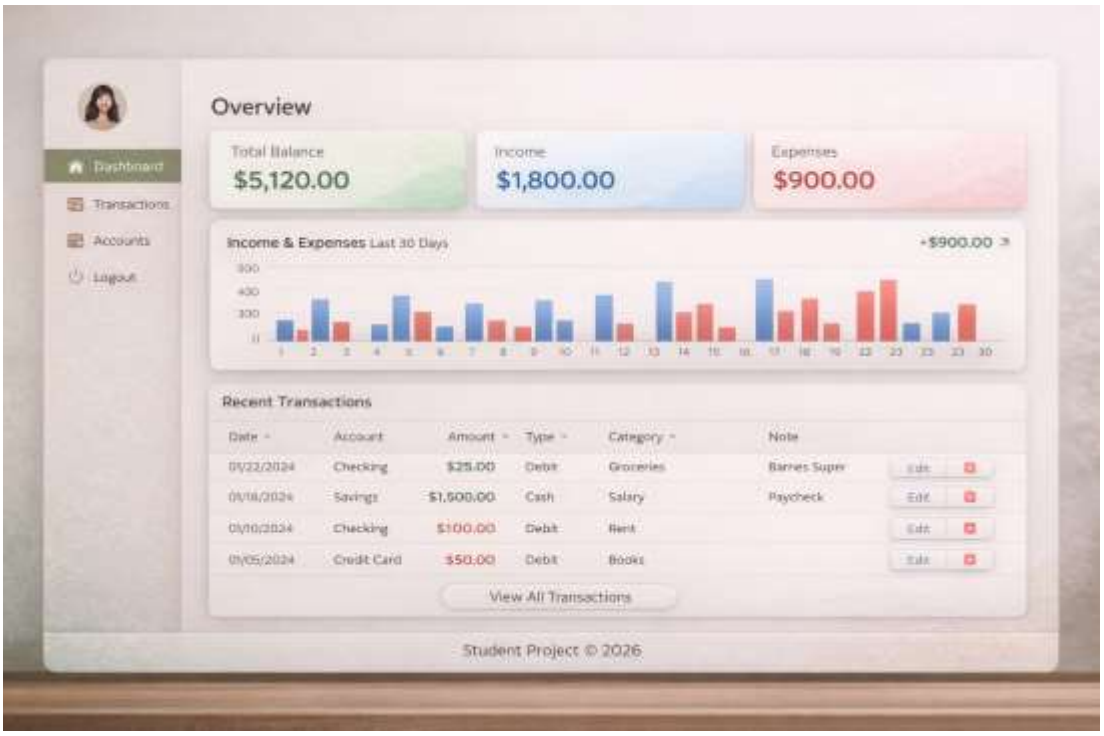
User will be able to manage multiple financial accounts within the app.

UI Mock-Ups

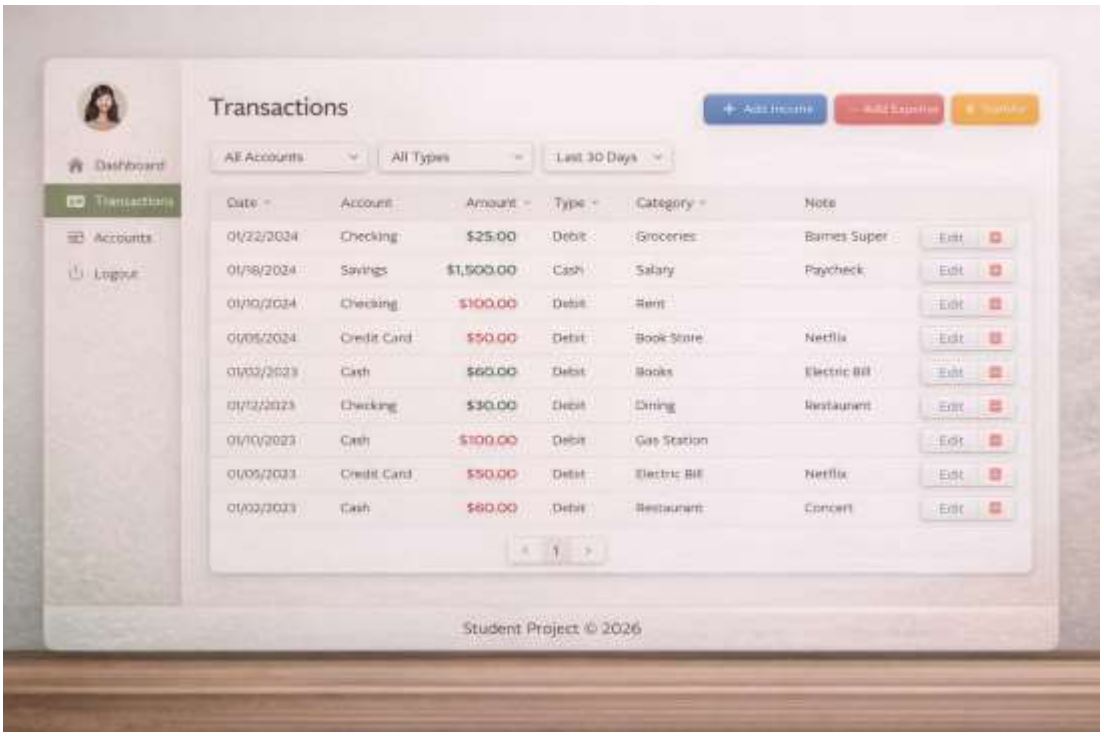
Pages (4): Dashboard, Transactions, Accounts, Account Detail
Additional Page(s): 404/Error

Note: Exclude transfers in v1.0 app design.

Dashboard

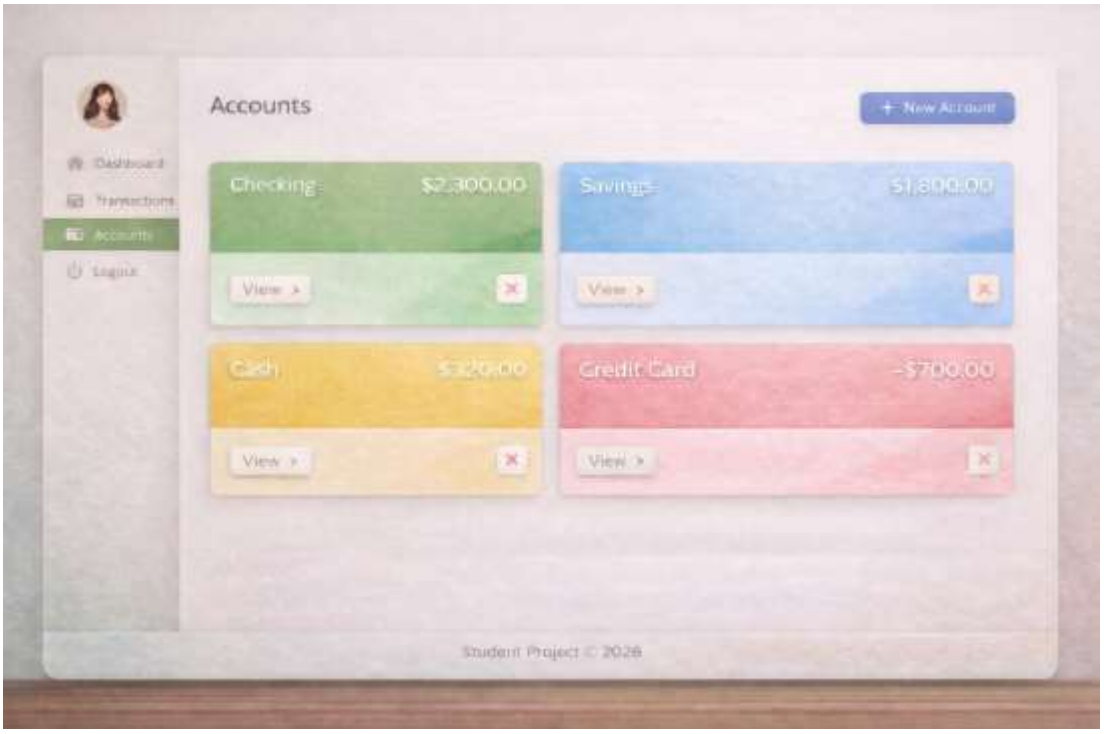


Transactions



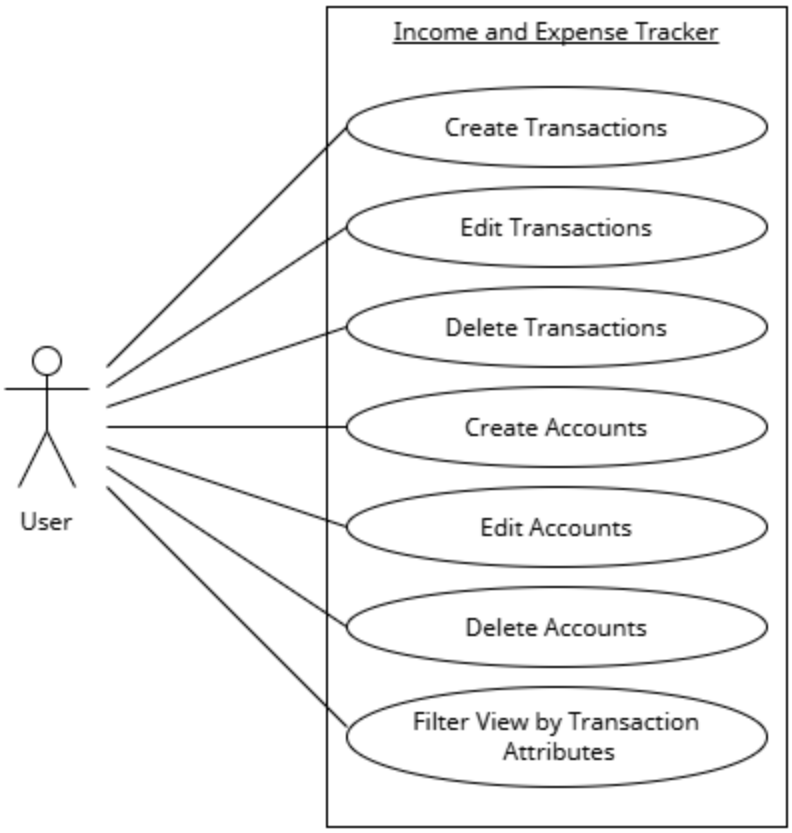
Accounts

Note: Transpose View and Delete, and ensure account cards are the same width.

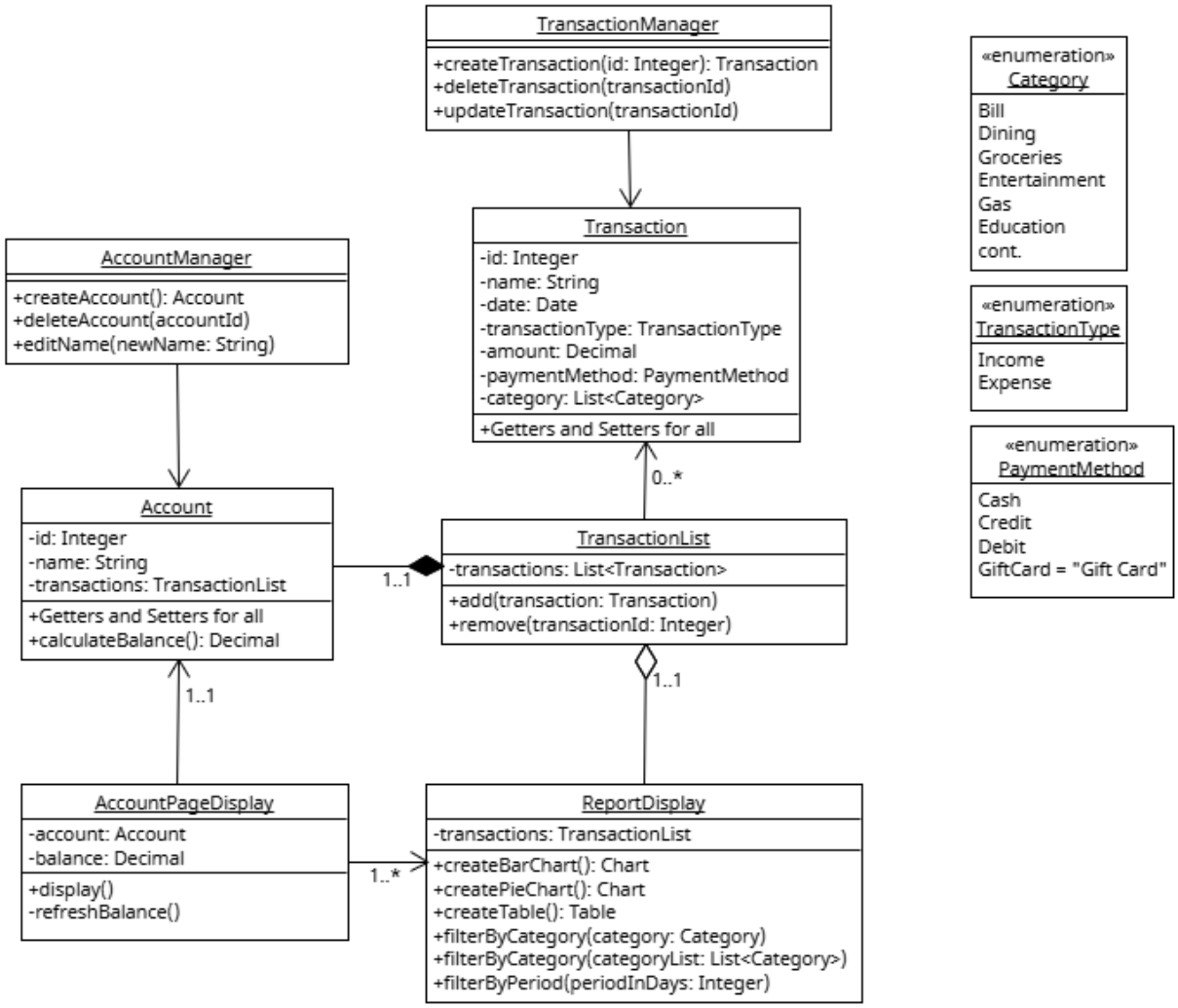


Account Details

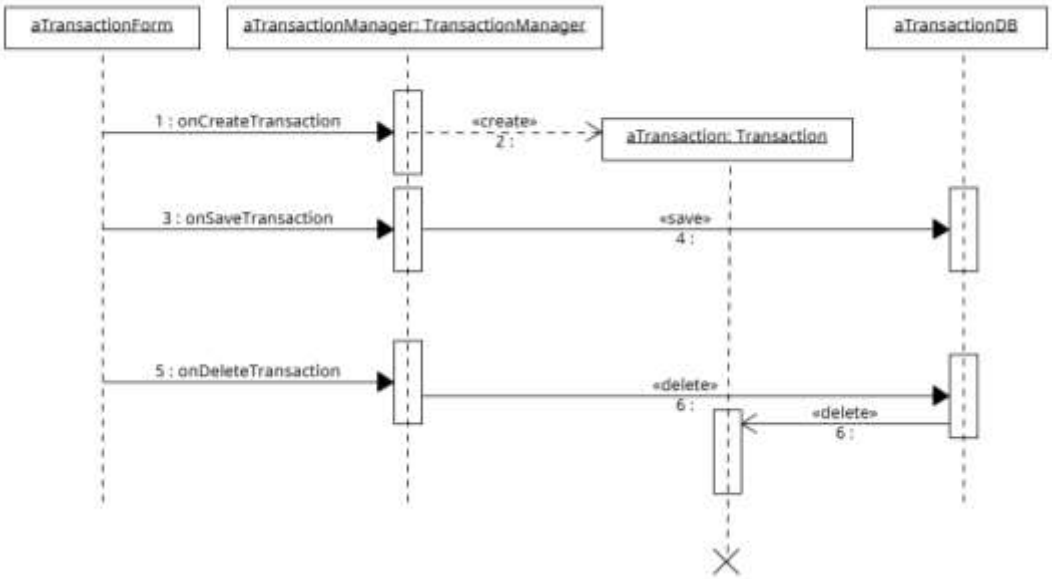
Use Case Diagram



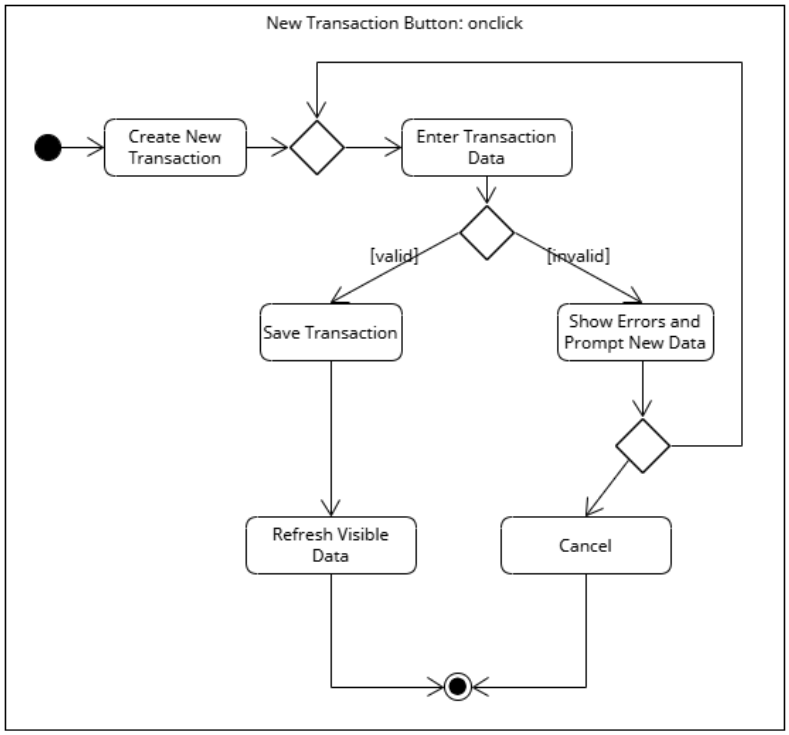
Class Diagram



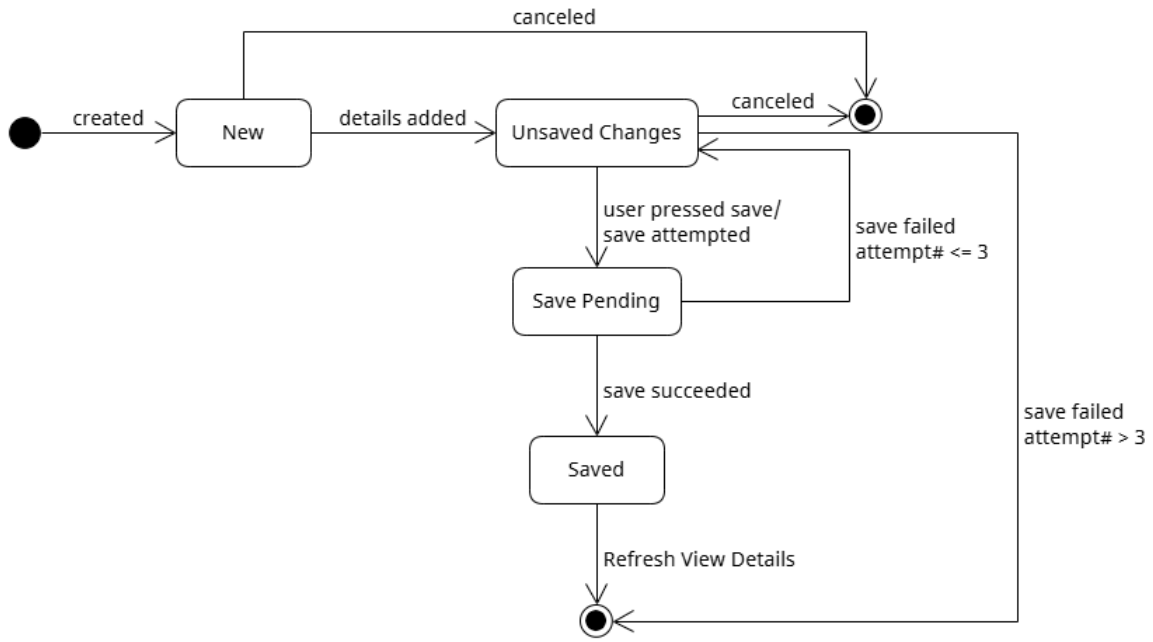
Sequence Diagram



Activity Diagram



State Diagram



Programs

Made using HTML, CSS, JS, and the NodeJS Library (potentially also the ExpressJS Library)

- root/
  - logs/
    - app.log
      - Logs all site & server errors and critical events
  - config/
    - config.js
      - Sets BASE_PATH and BASE_URL
    - dbconfig.js
      - Database connection data array
  - public/
    - styles/
      - general.css
        - CSS needed across entire website
    - scripts/
      - general.js
        - JavaScript needed across entire website
    - images/
      - placeholder-pfp.png
        - Image that currently resides in the account profile picture in the upper left corner of the navigation panel
  - routes/
    - index.js

            - Controls/centralizes all page links
- views/
    - includes/
        - footer.ejs
            - Footer for entire site, including copyright
        - navigation.ejs
            - Navigation panel for all pages, including a dummy profile picture and the links for Dashboard, Transactions, and Accounts
    - presenters/
            - account-presenter.ejs
            - HTML-generation code for the account details on both the account and the account-details pages
        - report-presenter.ejs
            - Methods for chart creation, table creation, and filter dropdown creation
    - pages/ *(All files have/generate the user-viewable page data—contains HTML and as needed, presenter method calls)*
        - error.ejs
            - Screen for when there is a 404 or other access-preventing error
        - dashboard.ejs
            - Overview of all account data, including total balance, income, expenses, a bar chart of income and expenses, and the recent transactions
            - ***Includes:*** *navigation.ejs and footer.ejs*
        - transactions.ejs
            - Shows 10 most recent expenses with buttons to move between visible expenses
            - Has filter options
            - Has buttons to add income or expense
            - ***Includes:*** *navigation.ejs and footer.ejs*
        - accounts.ejs
            - Has panels for each account with their name, current balance, a view button, and a delete button
            - Has 'New Account' button
            - ***Includes:*** *navigation.ejs and footer.ejs*
        - account-detail.ejs
            - Shows balance and has filters
            - Shows income and expenses over the user-selected period (last 30 days by default)
            - Has spending breakdown pie chart by category
            - Has bar chart for income and expenses over the period
            - Has table of transactions
            - Selected account names are on the website URL
                - ex: domain.com/account-detail/accountname1+accountname
                - May be where the backend receives the selected account
            - ***Includes:*** *navigation.ejs and footer.ejs*
- controllers/
    - middlewares/
        - helper.js
            - Sanitization functions
        - transaction-validation.js

            - Functions to validate (user-provided) transaction creation, update, and deletion data, returning true or false with a list of errors
        - account-validation.js
            - Functions to validate (user-provided) account creation, update, and deletion data, returning true or false with a list of errors
    - transaction-controller.js
        - Receives HTTPs requests in **'(req, res)'** functions
        - Validates, then sanitizes, then calls Services—all contained in try-catches, promises, and/or if-errors for error handling
        - On success, has the presenters and pages render updated version of the page that sent the request, potentially including a pop up telling the user it was a success
        - On failure, re-renders page with error messages in the form
    - account-controller.js
        - Receives HTTPs requests in **'(req, res)'** functions
        - Validates, then sanitizes, then calls Services, all contained in try-catches, promises, and/or if-errors for error handling
        - On success, has the presenters and pages render updated version of the page that sent the request, potentially including a pop up telling the user it was a success
        - On failure, re-renders page with error messages in the form
- models/
    - database/
        - database.js
            - Database connection class
        - transaction-db.js
            - Transaction SQL CRUD class
        - account-db.js
            - Account SQL CRUD class
    - orm-services/
        - transaction-manager.js
            - TransactionManager class
        - account-manager.js
            - AccountManager class
    - domain-entities/
        - transaction-list.js
            - TransactionList class
        - transaction.js
            - Holds Transaction class
        - account.js
            - Holds Account class
        - enums/
            - category.js
                - What the money was spent on/came from
            - transaction-type.js
                - Income, Expense
            - payment-method.js
                - Cash, Credit, Debit, Gift Card
- app.js

- Entry point for Node.js server
- Sets up server, initializes middleware, and mounts routes

Conclusion

This document presented the design of a personal monetary tracking application using UML class diagrams to illustrate the structure of the app and its component relationships. The design provides a foundation to guide the future development of the app.